

AGV Fleet Monitor 3D & Digital Twin



A Cyber-Physical System Integrating Edge Microcontrollers, Real-Time WebSockets, and Three.js 3D Virtualization.

PROJECT: Physical AI - ABA Fusion (Semaine 2)

FINAL DELIVERY: Jour 4 (Technical Report)

DEVELOPED BY: Hafida Belayd & Abdelkhalek Hanbel



1. Executive Summary

Modern warehouse automation relies extensively on Automated Guided Vehicles (AGVs) to coordinate logistics. Managing a physical fleet of AGVs presents challenges, particularly around system visibility, network latency, and collision prevention. This project resolves these issues by implementing a cyber-physical digital twin architecture.

By combining an embedded ESP32 microcontroller, a Python simulation routing broker, and an interactive React Three Fiber 3D client, we provide operators with real-time telemetry streaming, visual route coordinates, and manual trajectory overrides. Security is enforced via a two-layer safety mechanism combining automated hardware interrupts and cloud coordinated software locks.

2. System Architecture

The system uses a four-layered Cyber-Physical Architecture designed to ensure decoupling, reliability, and real-time visualization. Data flows from sensors up to the browser visualizer and down from the user HUD as commands.

SYSTEM ARCHITECTURE (DIAGRAMME D'ARCHITECTURE)

Embedded Edge Layer (ESP32 Microcontroller)

Reads HC-SR04 ultrasonic sensor, controls L298N motor driver, updates local I2C LCD displaying custom status icons.

↕ (SSL MQTT Port 8883 - JSON/Text Payloads)

Communication Bridge (HiveMQ Cloud Broker)

Brokers telemetry topics ("twin2/telemetry") and command queues ("robot/control") over secure TLS.

↕ (Background Telemetry Queue Worker Threads)

Simulation & Routing Engine (Python Backend Server)

Computes Dijkstra path planning, manages Zone X Mutex lock, hosts WebSocket server, and generates Moroccan Arabic (Darija) TTS audio via `ar-MA-MounaNeural`.

↓ (10Hz WebSocket JSON Broadcast + Base64 MP3 Speech Payloads)

3D Presentation HUD (Vite + React + R3F)

Renders animated AGVs with rotating LIDARs and dynamic pointLights, displays glassmorphic status controls, and plays Darija voice announcements via Web Audio API.

3. Embedded Edge Layer (ESP32 Firmware)

The physical vehicle is governed by an ESP32 edge microcontroller executing a non-blocking scheduling loop. The firmware maintains strict timing for telemetry updates, LCD refreshes, ultrasonic distance scans, and MPU6050 gyroscope tracking to maximize responsiveness.

3.1. Non-Blocking Design, MPU6050 Gyroscope, and LCD Layout

Unlike legacy looping structures that rely on `delay()`, our firmware uses `millis()` timer comparison thresholds. This ensures the MQTT callback and loop processing run at full speed.

We integrated an **MPU6050 6-Axis Gyroscope** to track the vehicle's precise Z-axis (Yaw) rotation in real-time. This provides highly accurate heading data via the `"robot/angle"` MQTT topic, which is used to execute perfect one-shot turns (e.g., 90° or 180°) without relying on blind, inaccurate time delays. The local I2C LCD screen (1602) simultaneously displays these live operational metrics.

To elevate visual feedback, we registered custom character binary matrices directly to the LCD memory during ``setup()`` to render icons:

-  (Robot icon) displayed next to active speed values.
-  (Battery symbol) displayed next to battery levels.
-  (Heartbeat symbol) that blinks dynamically every 500ms when connected to the broker.
-  (Hazard symbol) that flashes rapidly at 250ms intervals if an obstacle is detected in the safety range.

3.2. Startup Animation

During the initialization phase inside ``setup()``, the LCD runs a visual loading sequence. It prints a progress bar block-by-block across the second line (16 segments total) and displays `"Booting AGV OS... "` before flashing `"System Active! "`.

4. Software & Coordination Layer (Python Backend)

The Python server (running `headless_simulator.py`) manages fleet physics and navigation logic. It handles path routing and coordinates traffic intersections to prevent collisions.

4.1. Dijkstra Path Planning

The warehouse is defined as a graph containing node coordinates (Zones A, B, C, D, Charging Zone R) and lanes. When a new target is dispatched, the backend computes the shortest path using Dijkstra's algorithm and streams coordinates step-by-step.

4.2. Traffic Coordination (Zone X Mutex)

Zone X is a central intersection prone to collisions. Access is governed by a software ****Mutex Lock**** (`ZoneXTrafficController`). If AGV-01 occupies the zone, a lock is acquired, and any secondary AGVs requesting access are placed in a `WAIT` state. Once AGV-01 exits, the lock is released, and the next vehicle proceeds.

4.3. Dynamic Pathfinder & Obstacle Avoidance (Dijkstra Re-routing)

When a vehicle (physical or simulated) encounters an obstacle, it reports the incident to the Python backend server. The server immediately closes this lane segment programmatically, triggering a real-time recalculation of the path using Dijkstra's algorithm. This closes the edge-to-server control loop, rerouting the vehicle safely around the obstacle.

5. Real-Time 3D Digital Twin Visualizer

The React frontend connects via WebSockets to the Python backend to synchronize the virtual and physical states. It features an interactive ****Glassmorphism HUD**** panel built with CSS backdrop-filters, custom Outfit/Space Grotesk typography, and neon glows.

Inside the Three.js 3D canvas (React Three Fiber), the visual assets have been enhanced to match the physical states:

- **Spinning LIDAR:** A 3D LIDAR cylinder is rendered on top of the AGV model and spins continuously using the animation loop.
- **Dynamic Status LEDs:** An LED emitter changes colors based on the AGV's state: Green (moving), Orange (waiting/yielding), Red (emergency stop), and Rose (critical intersection).
- **Floor PointLight Projection:** A PointLight projects status-colored light on the floor grid under the AGV, creating real-time visual shadows.
- **Emissive Nodes:** Blue glowing spheres indicate junction coordinates.
- **First-Person View (FPV) Camera:** Operators can toggle FPV camera tracking, positioning the view slightly behind and above the AGV group, oriented dynamically to face the vehicle's driving direction.
- **Pulsing Blocked Lanes & Warning Signs:** Path segments marked as blocked render as pulsing, semi-translucent red hazard ribbons on the floor grid with 3D warning triangles that automatically face the camera.
- **Moroccan Arabic (Darija) Neural TTS Alerts:** All voice announcements are generated server-side using Microsoft Edge TTS (`ar-MA-MounaNeural` neural voice), encoded as Base64 MP3, and streamed through the WebSocket for browser playback. This replaces the unreliable browser-level `speechSynthesis` API, delivering consistent, high-quality Moroccan Arabic speech for all fleet events.

5.1. Darija TTS Technical Flow

When any fleet event occurs (E-Stop, dispatch, obstacle, Zone X entry, etc.), the Python backend calls the `announce(text)` coroutine. This function invokes `generate_darija_speech(text)`, which uses the `edge_tts.Communicate` API to synthesize an MP3 file using the `ar-MA-MounaNeural` Microsoft voice. The audio bytes are Base64-encoded and broadcast to all connected WebSocket clients as a special `{"type": "speech", "audio": "..."}` payload. The React frontend detects this payload type in its `ws.onmessage` handler and plays it using `new Audio("data:audio/mp3;base64," + payload.audio).play()`.

5.2. Autonomous "Square Runner" Mission

The 3D Dashboard features a dedicated **"Auto Mission"** system capable of autonomous sequence execution. We implemented a **"Square Runner"** sequence that drives the vehicle in a perfect square by looping four times through a configurable forward driving period and a rotation period. The progress of these maneuvers is dynamically visualized in the HUD via animated progress bars and countdown timers synchronized with the physical robot's movements.

DARIJA VOICE EVENTS REFERENCE

-  **E-Stop Activated:** "توقيف الطوارئ تخدم، الأسطول كامل وقف"
-  **E-Stop Cleared:** "حيد توقف الطوارئ، الأسطول رجع يخدم"
-  **Fleet Paused:** "الأسطول موقوف دابا"
-  **Fleet Resumed:** "الأسطول رجع يخدم دابا"
-  **Zone Dispatch:** "[Zone] العربية آ جي في واحد عادة دابا لمنطقة"
-  **Zone X Entry:** "رد البال، العربية دخلت لمنطقة التقاطع الخطيرة إكس"
-  **Obstacle Detected:** "حضي راسك، كاين عائق قدام العربية. جاري إعادة حساب مسار جديد"
-  **Lane Re-routing:** "كاين طريق مقطوعة. جاري إعادة حساب مسار جديد باستعمال ديكسترا"
-  **Obstacles Cleared:** "تمت إزالة الحواجز والعوائق، الطريق دابا مسرحة"
-  **Voice Test:** "فحص الصوت خدام مزيان، النظام دابا أونلاين"

6. Cyber-Physical Sequence & Data Flow

The diagram below traces the communication lifecycle when an operator triggers a route dispatch, highlighting the edge-level safety interrupts.

SYSTEM SEQUENCE FLOW (FLUX DE DONNÉES)

Route Dispatch

1

Operator clicks Zone A on React HUD. Web client sends `{"command":"dispatch","target":"A"}` via WebSocket to Python backend.

Path Planning

2

Python server runs Dijkstra's algorithm to compute shortest path and starts updating motion steps.

Hardware Instruction

3

Python publishes command string (e.g. `"FORWARD"`) to the HiveMQ MQTT broker on topic `"robot/control"`.

Edge Execution & Scan

4

ESP32 receives command via Wi-Fi and triggers motor pins, while checking proximity using the HC-SR04 sensor.

Edge Safety Interrupt (Distance < 20cm)

5

If an obstacle is detected, ESP32 overrides motor speed to 0 immediately, flashes `"⚠"` on I2C LCD, and publishes emergency telemetry to MQTT.

Digital Twin Sync

6

Python backend reads MQTT warning telemetry, stops simulated motion, and broadcasts `emergency_stop: true` to React client.

Dashboard Visual Warning

7

React displays red flashing alert overlay. Virtual AGV status LED shifts to Red, casting a red `PointLight` glow on the grid.

Dynamic Dijkstra Re-routing

8

Python backend registers the obstacle, programmatically closes the lane, notifies the React client (which renders the lane red with 3D warning flags), and dynamically recalculates a Dijkstra route to guide the vehicle around the blocked area.

Moroccan Arabic (Darija) Neural TTS Announcement

9

Python backend generates MP3 audio using `ar-MA-MounaNeural` Edge TTS voice, encodes as Base64, and broadcasts as `{"type":"speech","audio":"..."}` payload. Browser plays the audio automatically via Web Audio API.

7. Security and Safety Failsafes

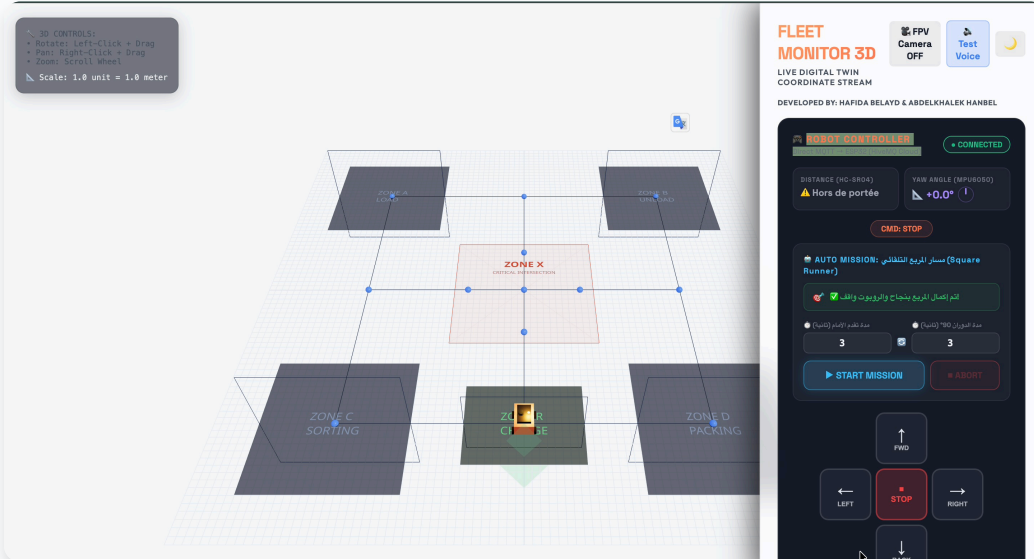
CRITICAL SAFETY SYSTEMS

To prevent runaway movement, the ESP32 implements a **1500ms Watchdog Timer**. If the Python backend fails to send commands within this window, the ESP32 automatically halts all motors. Furthermore, if Wi-Fi or MQTT connections are lost, the robot cuts motor power instantly before executing reconnect logic.

8. System Interface & Visual Analysis

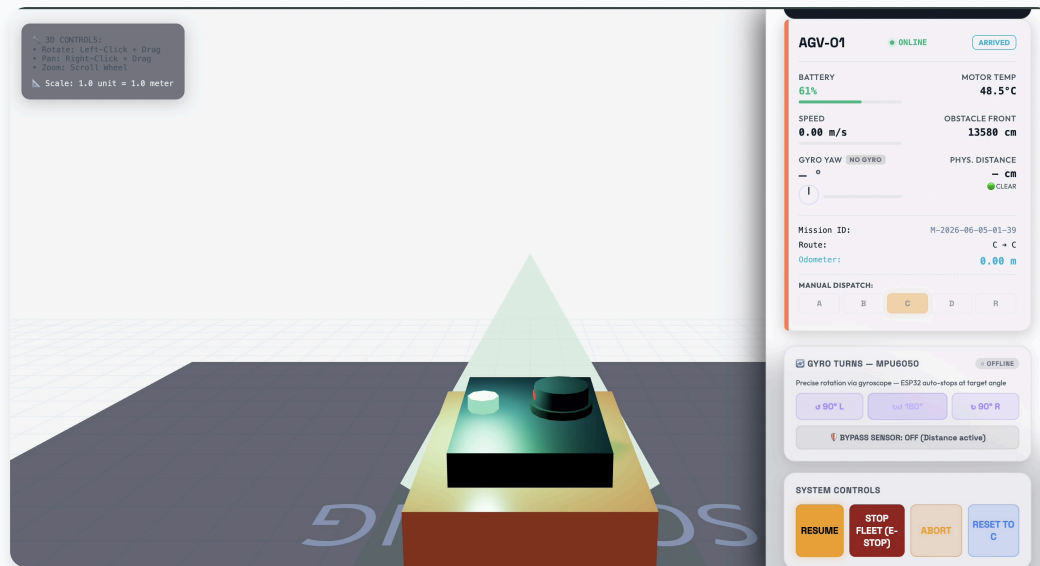
The following snapshots demonstrate the various operational states of the 3D Digital Twin and Fleet Monitor interface during real-time testing.

1. Square Runner Autonomous Mission



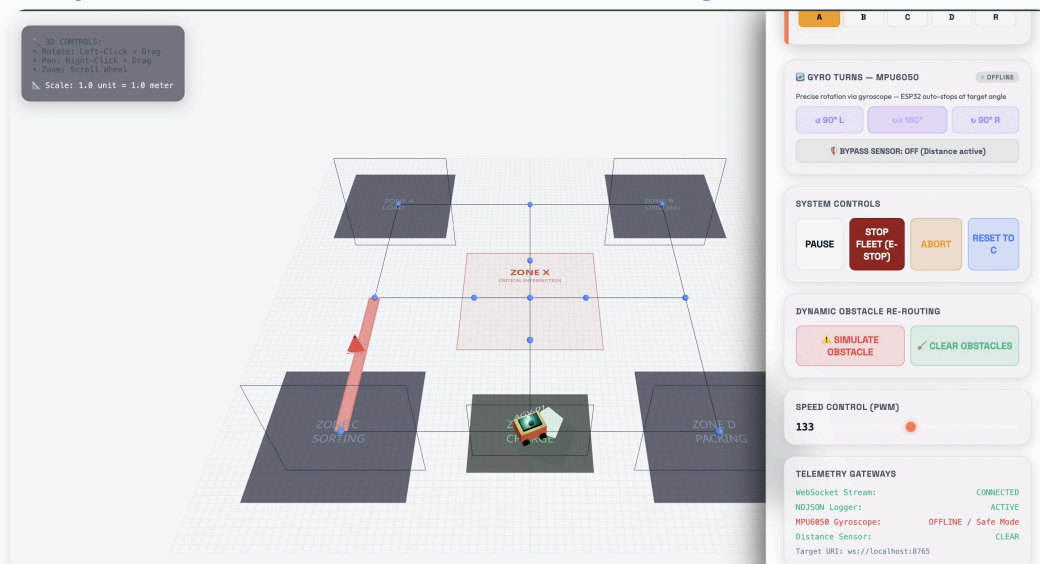
Analysis: This view highlights the "Auto Mission: Square Runner" execution pane. The interface shows a successfully completed autonomous square pattern with dynamic countdown timers synchronized for both Forward movement and 90° Gyroscope turns. The system correctly tracks time intervals mapped to real-world physics.

2. Telemetry and Manual Dispatch



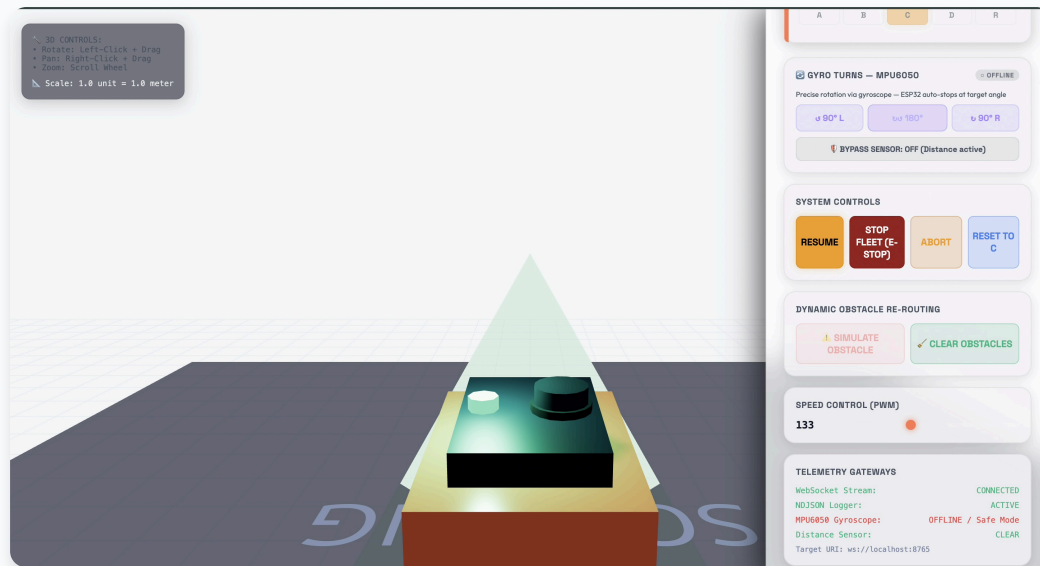
Analysis: The main Fleet Monitor layout displaying active telemetry. The sidebar tracks Live Battery (61%), Distance Sensor values, and precise Gyro Yaw angles. The 3D view shows the vehicle positioned at Zone C with realistic dynamic lighting and shadows.

3. Dynamic Obstacle Detection & Rerouting



Analysis: Shows the system's reaction to an obstacle. The route between Zone C and Zone X is dynamically marked in red (blocked), and the system highlights the active hazard using visual alerts on the 3D grid, ready to recalculate an alternate path using Dijkstra's algorithm.

4. FPV Camera System



Analysis: First-Person View (FPV) Camera mode activated. The camera tightly follows the AGV from behind, showing the rotating LIDAR cone and emissive LED lights from an operator's perspective on the ground.

9. Conclusion

By integrating edge hardware safety overrides with an interactive 3D virtual twin and a native Moroccan Arabic (Darija) Neural TTS voice system, this project delivers a highly secure, traceable, and accessible AGV fleet coordination platform. The low-latency MQTT/WebSocket synchronization, real-time visual warnings, and spoken Darija announcements powered by the `ar-MA-MounaNeural` Microsoft Edge TTS voice ensure safe and linguistically inclusive industrial automation for Moroccan operator environments.